

CS 230 Outline Midterm Review 2

Final Review CS230

Questions from Review Session:

Implementing tree traversal:

Do we use an iterator for arrays?

Hash table vs hash map

GOOGLE FORM RESPONSES

- Tree Traversals
- Outline of what could be on the exam
- Space and Time complexity, implementing hash tables and graphs
- How inorder, preorder, and postorder works
- How do different sorting algorithms work?
- Search (dfs, binary search, bfs)
- Implementing hash tables

TREES

Definitions & Applications

- What's a tree?
A connected acyclic graph with a single root node.
- Similarities/Differences with Graphs:
 - Tree is a special graph (no cycles, one root).
 - Graphs can have cycles and multiple roots.
- Applications:
File systems, parsers, decision trees, org charts.

Properties

- Subtree: A tree rooted at a child node.
- Path: Sequence of nodes from parent to child.
- Ancestor/Descendant: Parent/child relations.
- Level: Distance from root.
- Height: Longest path from root to a leaf.
- Binary tree: Max 2 children per node.

- n-ary tree: Can have n children.
- Balanced tree: Heights of subtrees differ by ≤ 1 .
- Complete tree: All levels filled except maybe last, filled left-to-right.
- Full tree: Every node has 0 or 2 children.

Decision Trees

- What are they?
Trees where each node represents a decision or condition; leaves are outcomes.
- Use cases:
Classification problems, rule-based systems, games.
- Implement/Traverse:
Traverse according to conditions until a leaf node is reached.

Binary Search Trees (BST)

- What are they?
Binary trees where $\text{left} < \text{root} < \text{right}$.
- Use cases:
Efficient lookups, sorted data, dynamic sets.
- Complexity:
 - **Balanced**: $O(\log n)$ for insert/search/delete
- Traversals: inorder gives sorted order.
- degenerate (like a linked list): **$O(n)$**

Explain how to implement tree methods and traversals recursively

Tree properties (balanced, full, complete)

Preorder - Visit node, traverse left, traverse right

Inorder - left, visit, right

Postorder - left, right, visit

Level-order - uses queue, visits all children first, similar to **BFS**

Give them a tree to traverse? One walk-through example together, then they try it by themselves in small groups?

Binary Search Trees - elements larger than root to the right, elements small to the left

Linked Tree vs. Array Tree Implementation

Degenerate binary trees and why they're bad: A grossly unbalanced tree, with some long paths

When does it occur?

Why is it undesirable?

SORTING AND SEARCHING

Selection Sort:

SelectionSort orders values by repeatedly putting a particular value into its final sorted position

The algorithm:

1. find the smallest value in the dataset
2. switch it with the value in the first position
3. find the next smallest value in the dataset
4. switch it with the value in the second position
5. repeat until all values are in their sorted positions

Insertion Sort:

InsertionSort orders a list of values by repeatedly inserting a particular value into a sorted subset of the list

The algorithm:

1. consider the first item to be a sorted sublist of length 1
2. insert the second item in the sorted sublist,
3. shifting the first item if needed
4. insert the third item into the sorted sublist,
5. shifting the other items as needed
6. repeat until all values inserted into their proper positions

Merge sort:

Merge sort orders a list of values by recursively dividing the list in half until each sublist has one element, then recombining (merging) the sublists

The algorithm:

Split: Divide the list into two equal parts—left and right. If the list has odd length, one half will have an extra element.

Sort: Recursively use merge-sort to sort the two lists. Note that by definition, a list of one element is sorted (base case).

Merge: Merge the two sorted parts into one sorted list

Algorithm	Best	Average	Worst	Space in addition to original array
-----------	------	---------	-------	-------------------------------------

Insertion	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$
Selection	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$
Merge	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$

Worst-case time complexities

Linear Search

Binary Search:

What is binary search?

A search algorithm that finds the position of a target value within a **sorted array** by repeatedly dividing the search interval in half.

How does it work?

1. Start with two pointers: `low` and `high`.
2. Find the middle: `mid = (low + high)/2`.
3. If `target == arr[mid]`, return index.
4. If `target < arr[mid]`, search left half.
5. If `target > arr[mid]`, search right half.
6. Repeat until found or interval is empty.

Runtime complexity:

- Time: $O(\log n)$
- Space: $O(1)$ (iterative), $O(\log n)$ (recursive)

HASH TABLES

- **What is a hash table?**
A data structure that maps keys to values using a **hash function**.
- **Use cases:**
Fast lookups, insertions, deletions (e.g., dictionaries, caches).

- **Hash function:**
Converts a key to an index. Good hash functions minimize collisions.
- **Good vs. Bad:**
 - *Good:* Uniform distribution.
 - *Bad:* Clusters values, many collisions.
- **Complexities:**
 - Average Time: **$O(1)$** for lookup/insert/delete.
 - Worst-case: **$O(n)$** (due to collisions).
 - Space: **$O(n)$**
- **Collision resolution:**
 - *Separate chaining:* Each index has a linked list of entries.
 - *Linear probing:* Find next open slot.
- **Load factor:**
Ratio of number of entries to table size.
 - Higher load $\rightarrow$ slower, more collisions.
 - Resize table when load > threshold (e.g., 0.75).

Explain time complexities

Linked vs linear probing implementation

Graphs

Directed VS Undirected:

Properties

- Subgraph: A subset of a graph's vertices and edges.
- Types of Paths:
 - *Simple path:* No repeated vertices.
 - *Cycle:* Starts and ends at same vertex.
 - *Simple cycle:* A cycle with no repeated vertices or edges (except start/end).
- Complete graph: Every vertex is connected to every other vertex.
- Strongly connected graph: A directed graph where every vertex is reachable from every other.
- Strongly connected component (SCC): Maximal subgraph that is strongly connected.

Implementations:

- Adjacency Matrix (2D array):
- Adjacency List

Traversal Algorithms: BFS & DFS

BFS (Breadth-First Search):

- Uses a **queue**, explores level-by-level.
- Useful for **shortest path**, checking connectivity.

DFS (Depth-First Search):

- Uses a **stack** (explicit or recursive), goes deep first.
- useful for **cycle detection**, **topological sort**, etc.

Simulate by hand: Draw graph, mark visited nodes, trace stack/queue contents.

Return path / reachable vertices:

WORKSHEETS

 Copy of Copy of SP23 21_SearchingSorting_Blank

 Copy of Copy of SP23 21_SearchingSorting_WrittenSolution.pdf

 Copy of Copy of SP23 20_HashTable_Blank

 Copy of Copy of SP23 17_Analysis_Blank

 Copy of Copy of SP23 18_DFS_Blank

 Copy of Copy of SP23 23_TreeImplementations_Blank